

CoolPeople — Remaining §4/§5/§6 backend: tables & files reference

For the functions/routes you're building next. Each row shows the **tables** it touches (**R** = reads, **W** = writes) and the **file** it lives in. Shared reference tables (`jurisdiction` , `office` , `rac`es , `users`) are read by many for joins/keys. **[DB]** marks rows added by the Debate Update (migration `1780700000000`).

Conventions: DB logic in `server/DB/<feature>.js` , routes in

`server/API/<feature>Routes.js` , each fn takes a single `{ ... }` object and

returns rows. `requireAuth` (A) / `requireAdmin` (AD) / `recordAdminAction` (ADA)

/ `requireAttestation` (ATT) / `requireAgeGate` (AGE) / `requireCriteriaAck` (ACK)

/ `captureRequestContext` (CTX) / `requireInternal` (internal) live in `server/middleware` .

Status: ☒ built · ☐ to build.

§4 — FEC / Election-Law Compliance Gates — none built

The legal layer that decides whether a WouldBe may go live and solicit. Nothing here transacts money; it gates `launch_status` and records why.

1. Candidate committees

Route file: `server/API/candidateCommitteesRoutes.js` · **DB file:** `server/DB/candidateCommittees.js` (new) **Tables in play:** `candidate_committees` (+ `filing_authorities` , `jurisdiction` , `office` , `users` for joins)

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/committees</code>	<code>createCandidateCommittee</code> (<i>treasurer fields + filing receipt → registration_status='provisional_on_receipt'</i>)	<code>candidate_committees</code> W · <code>filing_authorities</code> R	A, ATT	☐
GET <code>/committees/me</code>	<code>getUserCommittees</code>	<code>candidate_committees</code> R	A	☐
GET <code>/committees/:id</code>	<code>getCommitteeById</code>	<code>candidate_committees</code> R	A	☐
POST <code>/committees/:id/verify</code>	<code>verifyCommitteeViaAPI</code> · <code>confirmCommittee</code> (<i>provisional → verified_active ; sets verified_via_api / last_verified_at</i>)	<code>candidate_committees</code> R/W	A	☐
PATCH <code>/committees/:id</code>	<code>updateCommittee</code>	<code>candidate_committees</code> W	A	☐
(launch gate)	<code>hasActiveVerifiedCommittee</code> (userId, raceId) (<i>accepts provisional_on_receipt</i>)	<code>candidate_committees</code> R	—	☐

This is the launch gate — §5's `createWouldbe` / `updateWouldbe` can only flip `launch_status` to `active` once `hasActiveVerifiedCommittee` passes.

Column cheat-sheet

- `candidate_committees` : id, user_id, jurisdiction_id, committee_name, committee_type, external_committee_id, external_candidate_id, office_sought, office_district, cycle_year, registration_status, verified_via_api, verification_response, last_verified_at, registration_date, termination_date, filing_authority_id, treasurer_name, treasurer_relationship, is_self_treasurer, filing_receipt_url, filing_receipt_number, filed_at, committee_id_status, created_at

2. Compliance checks

Route file: `server/API/complianceChecksRoutes.js` · **DB file:** `server/DB/complianceChecks.js` (new) **Tables in play:** `compliance_checks` (+ `jurisdiction_rules_versions` , `user_jurisdictions` , `wouldbe`)

Method / Path	Function	Tables (R/W)	MW	Status
POST /compliance-checks/run	runComplianceCheck (auto-resolves jurisdiction via user_jurisdictions ; stamps jurisdiction_rules_version)	compliance_checks W · jurisdiction_rules_versions R · user_jurisdictions R	A	☐
GET /compliance-checks/me	getChecksForUser	compliance_checks R	A	☐
GET /wouldbes/:id/compliance	getChecksForWouldbe · getLatestCheckResult	compliance_checks R	A	☐

Column cheat-sheet

- compliance_checks : id, user_id, wouldbe_id, check_type, jurisdiction_id, jurisdiction_rules_version, result, reason, details, external_verification_payload, performed_at, performed_by, performed_by_user_id

3. Jurisdiction rules (versioned law)

Route file: server/API/jurisdictionRulesRoutes.js · **DB file:** server/DB/jurisdictionRules.js (new) **Tables in play:** jurisdiction_rules_versions (+ jurisdiction)

Method / Path	Function	Tables (R/W)	MW	Status
GET /jurisdiction-rules/:jid/current	getCurrentRulesForJurisdiction (effective_until IS NULL)	jurisdiction_rules_versions R	—	☐
GET /jurisdiction-rules/:jid/history	getRulesHistory · getRulesAtPointInTime	jurisdiction_rules_versions R	—	☐
POST /admin/jurisdiction-rules	publishRulesVersion (closes prior: sets effective_until)	jurisdiction_rules_versions W	AD, ADA	☐
(gate helper)	getAdvancementRule(jurisdictionId) (top-two / RCV / no-runoff via advancement_rule / advancement_threshold)	jurisdiction_rules_versions R	—	☐

Column cheat-sheet

- jurisdiction_rules_versions : id, jurisdiction_id, version, effective_from, effective_until, candidacy_trigger_type, candidacy_trigger_value, contribution_limit_individual_primary, contribution_limit_individual_general, contribution_limit_aggregate, committee_required_before_solicitation, off_session_fundraising_ban, matching_funds_program, matching_funds_rules_ref, contest_registration_threshold, has_runoff, advancement_rule, advancement_threshold, source_documents, created_by_user_id, created_at

4. Testing the Waters

Route file: server/API/testingTheWatersRoutes.js · **DB file:** server/DB/testingTheWaters.js (new) **Tables in play:** testing_the_waters_campaigns (+ candidate_committees on convert)

Method / Path	Function	Tables (R/W)	MW	Status
POST /ttw	startTestingTheWaters	testing_the_waters_campaigns W	A, ATT	☐
GET /ttw/me	getUserTTW	testing_the_waters_campaigns R	A	☐
GET /ttw/:id	getTTWById	testing_the_waters_campaigns R	A	☐
POST /ttw/:id/convert	convertTTWtoCommittee (sets converted_to_committee_id)	testing_the_waters_campaigns W · candidate_committees W	A	☐
POST /ttw/:id/terminate	terminateTTW	testing_the_waters_campaigns W	A	☐
(internal)	incrementTTWPledgeStats · checkTTWCaps (cumulative_pledges_cents vs soft / hard_pledge_cap_cents)	testing_the_waters_campaigns R/W	internal	☐

Column cheat-sheet

- testing_the_waters_campaigns : id, user_id, office_being_explored, jurisdiction_id, status, start_date, expiration_date, soft_pledge_cap_cents, hard_pledge_cap_cents, cumulative_pledges_cents, unique_pledgers_count, is_publicly_discoverable, disclosure_shown_to_pledgers, disclosure_version_id, converted_to_committee_id, expired_at, created_at

5. Fundraising eligibility

Route file: `server/API/fundraisingEligibilityRoutes.js` · **DB file:** `server/DB/fundraisingEligibility.js` (new) **Tables in play:** `fundraising_eligibility_checks` (+ `jurisdiction_rules_versions`, `user_jurisdictions`)

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/fundraising-eligibility/run</code>	<code>runFundraisingEligibilityCheck</code>	<code>fundraising_eligibility_checks</code> W · <code>jurisdiction_rules_versions</code> R	A	☐
GET <code>/fundraising-eligibility/me</code>	<code>getUserFundraisingChecks</code>	<code>fundraising_eligibility_checks</code> R	A	☐
GET <code>/fundraising-eligibility/:jid/latest</code>	<code>getLatestEligibilityForJurisdiction</code>	<code>fundraising_eligibility_checks</code> R	—	☐

Column cheat-sheet

- `fundraising_eligibility_checks` : `id`, `user_id`, `jurisdiction_id`, `office_sought`, `election_cycle`, `check_date`, `legal_status`, `committee_required`, `required_filings`, `legal_minimum_lead_days`, `recommended_lead_days`, `max_individual_contribution_cents`, `warnings`, `check_passed`, `created_at`

§5 — WouldBe Campaigns, Plans, Pledges, Follows — *partially built*

`wouldbe.js` and the staged-timeline splitter (`planTimeline.js`) have built functions (✓); plans/goals/pledges/follows are still to build.

6. WouldBe campaigns

Route file: `server/API/wouldbeRoutes.js` · **DB file:** `server/DB/wouldbe.js` (partially built) **Tables in play:** `wouldbe` (+ `office`, `racess`, `users`, `pledges`, `posts`; [DB] `wouldbe_creation_payments`, `pledge_goal_notifications`)

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/wouldbes</code>	<code>createWouldbeV2</code> (<i>launch_status='draft' ; enforce \$5K-\$1M</i>)	<code>wouldbe</code> W	A, ATT	✓
[DB] POST <code>/wouldbes/:id/creation-payment</code>	<code>recordWouldbeCreationPayment</code> (<i>\$5 fee → stamps creation_fee_paid_at</i>)	<code>wouldbe_creation_payments</code> W · <code>wouldbe</code> W	A	✓
[DB] GET <code>/wouldbes/:id/creation-payment</code>	<code>getWouldbeCreationPayment</code>	<code>wouldbe_creation_payments</code> R	A	✓
GET <code>/wouldbes</code>	<code>listWouldbes</code>	<code>wouldbe</code> R	—	✓
GET <code>/wouldbes/:id</code>	<code>getWouldbeById</code>	<code>wouldbe</code> R	—	✓
PATCH <code>/wouldbes/:id</code>	<code>updateWouldbe</code> (<i>enforce \$5K-\$1M</i>)	<code>wouldbe</code> W	A	✓
[DB] PATCH <code>/wouldbes/:id/processor</code>	<code>setContributionProcessor</code> (<i>ActBlue/WinRed link</i>)	<code>wouldbe</code> W	A	✓
POST <code>/wouldbes/:id/retire</code>	<code>retireWouldbe</code>	<code>wouldbe</code> W	A	✓
GET <code>/wouldbes/:id/pledgers</code>	<code>getWouldbePledgers</code>	<code>pledges</code> R · <code>users</code> R	A	✓
GET <code>/wouldbes/:id/posts</code>	<code>getWouldbePosts</code>	<code>posts</code> R · <code>wouldbe</code> R	—	✓
GET <code>/wouldbes/:id/rankings</code>	<code>getWouldbeRankings</code> (<i>standing vs same race/office</i>)	<code>wouldbe</code> R	—	✓
GET <code>/wouldbes/recommended</code>	<code>getRecommendedWouldbes</code> (<i>reuse notifier criteria + user_jurisdictions</i>)	<code>wouldbe</code> R · <code>user_jurisdictions</code> R	A	☐
[DB] (internal)	<code>recordGoalReached</code> (<i>goal/micro-goal met → processor link to pledgers; idempotent</i>)	<code>pledge_goal_notifications</code> W · <code>pledges</code> R	internal	✓
(internal)	<code>incrementPledgedTotal</code> · <code>checkWouldbeCanPostVideos</code>	<code>wouldbe</code> R/W · <code>pledges</code> R	internal	☐

Column cheat-sheet

- `wouldbe` : `id`, `title`, `description`, `user_id`, `office_id`, `race_id`, `goal_cents`, `pledged_total_cents`, `deadline`, `can_post_videos`, `retired`, `retired_at`, `entry_path`, `launch_status`, `approval_method`, `contest_external_ids`, `pledge_cap_cents`, `cap_reset_count`, `primary_advanced_at`, [DB] `creation_fee_cents`, `creation_fee_paid_at`, `contribution_processor`, `contribution_processor_url`, `created_at`, `updated_at`

- [DB] `wouldbe_creation_payments` : id, wouldbe_id, user_id, amount_cents, currency, stripe_customer_id, stripe_payment_intent_id, stripe_charge_id, stripe_balance_txn_id, fee_amount_cents, net_amount_cents, status, failure_reason, charged_at, refunded_at, created_at
- [DB] `pledge_goal_notifications` : id, wouldbe_id, plan_timeline_component_id, goal_kind, threshold_cents, reached_at, processor_url, pledgers_notified_count, notified_at, created_at
- `wouldbe_recommendations` (*nominate-someone*; §8): id, recommender_user_id, target_user_id, office_id, race_id, source_debate_id, recommendation_type, reason_text, target_response, responded_at, resulting_wouldbe_id, created_at

7. Plans

Route file: `server/API/plansRoutes.js` · **DB file:** `server/DB/plans.js` (*new*) **Tables in play:** `plan` , `plan_components` , `plan_component_categories` (+ `wouldbe` , `office`)

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/wouldbes/:id/plan</code>	<code>createPlan</code>	<code>plan</code> W	A	☐
GET <code>/wouldbes/:id/plan</code>	<code>getPlanForWouldbe</code>	<code>plan</code> R · <code>plan_components</code> R	—	☐
POST <code>/plans/:id/components</code>	<code>addPlanComponent</code>	<code>plan_components</code> W	A	☐
PATCH <code>/plan-components/:id</code>	<code>updatePlanComponent</code>	<code>plan_components</code> W	A	☐
DELETE <code>/plan-components/:id</code>	<code>deletePlanComponent</code>	<code>plan_components</code> W	A	☐
GET <code>/plan-categories</code>	<code>listPlanCategories</code>	<code>plan_component_categories</code> R	—	☐
POST <code>/admin/plan-categories</code>	<code>createPlanCategory</code>	<code>plan_component_categories</code> W	AD	☐

Column cheat-sheet

- `plan` : id, wouldbe_id, user_id, office_id, created_at, updated_at
- `plan_components` : id, plan_id, category_key, title, description, created_at
- `plan_component_categories` : category_key, display_name, category_group, description, icon, sort_order, is_active, created_at

8. Plan timeline + staged gating (WB §7)

Route file: `server/API/planTimelineRoutes.js` · **DB file:** `server/DB/planTimeline.js` (*partially built*) **Tables in play:** `plan_timeline` , `plan_timeline_components` , `stage_proofs` , `stage_gate_transitions` , `election_deadlines` , `deadline_offset_rules`

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/plans/:id/timeline</code>	<code>createPlanTimeline</code> · <code>syncTimelineFromRace</code>	<code>plan_timeline</code> W · <code>aces</code> R	A	☐
GET <code>/plans/:id/timeline</code>	<code>getPlanTimeline</code>	<code>plan_timeline</code> R · <code>plan_timeline_components</code> R	—	☐
POST <code>/timelines/:id/components</code>	<code>addTimelineComponent</code>	<code>plan_timeline_components</code> W	A	☐
PATCH <code>/timeline-components/:id</code>	<code>updateTimelineComponent</code>	<code>plan_timeline_components</code> W	A	☐
POST <code>/wouldbes/:id/build-stages</code>	<code>buildStagedTimeline(planTimelineId)</code> (<i>cumulative back-loaded split into per-deadline sub-goals</i>)	<code>plan_timeline_components</code> W · <code>election_deadlines</code> R · <code>wouldbe</code> R	A	☒
GET <code>/wouldbes/:id/stages</code>	<code>getStagedTimeline</code> · <code>getStage</code>	<code>plan_timeline_components</code> R · <code>election_deadlines</code> R	—	☒
(<i>prereq, internal</i>)	<code>computeDeadlinesForOffice</code> (<i>populates election_deadlines the splitter reads</i>)	<code>election_deadlines</code> W · <code>deadline_offset_rules</code> R	internal	☒
POST <code>/stages/:id/proof</code>	<code>submitStageProof</code> → <code>pullAuthorityProof</code> (T1) · <code>ingestProofDocument</code> + <code>crossCheckProof</code> (T2) · <code>recordStageAttestation</code> (T3)	<code>stage_proofs</code> W	A, CTX	☐
(<i>internal/cron</i>)	<code>evaluateStageGate(componentId)</code> (<i>flips gate_state / proof_status ; append-only audit</i>) · <code>getGateAuditTrail</code>	<code>plan_timeline_components</code> W · <code>stage_gate_transitions</code> W · <code>stage_proofs</code> R	internal	☐

Column cheat-sheet

- `plan_timeline` : id, plan_id, office_id, race_id, created_at
- `plan_timeline_components` : id, plan_timeline_id, event_date, timeline_category, completed, completed_at, stage_number, sub_goal_cents, gate_state, proof_tier, verify_method, milestone_type, proof_status, election_deadline_id, created_at

- `stage_proofs` : id, plan_timeline_component_id, wouldbe_id, proof_tier, verify_method, milestone_type, status, source, authority_payload, uploaded_document_url, extracted_fields, cross_check_result, attestation_user_id, attestation_statement, submitted_at, verified_at, verified_by, created_at
- `stage_gate_transitions` : id, plan_timeline_component_id, wouldbe_id, from_state, to_state, triggered_by, source, stage_proof_id, notes, transitioned_at

9. Goals (WB §4)

Route file: `server/API/goalsRoutes.js` · **DB file:** `server/DB/goals.js` (new) **Tables in play:** `office_recommended_goals` , `goal_increase_requests` (+ `wouldbe` , `office`)

Method / Path	Function	Tables (R/W)	MW	Status
GET <code>/offices/:id/recommended-goal</code>	<code>getRecommendedGoalForOffice</code> (<i>cents, or null</i>)	<code>office_recommended_goals</code> R	—	☐
POST <code>/wouldbes/:id/goal-increase</code>	<code>createGoalIncreaseRequest</code> (<i>reason required, ≤ \$1M</i>)	<code>goal_increase_requests</code> W	A	☐
GET <code>/admin/goal-increases</code>	<code>listPendingGoalIncreases</code>	<code>goal_increase_requests</code> R	AD	☐
POST <code>/admin/goal-increases/:id/decide</code>	<code>decideGoalIncrease</code> (<i>on approve → update wouldbe.goal_cents + re-scope timeline</i>)	<code>goal_increase_requests</code> W · <code>wouldbe</code> W	AD, ADA	☐

Column cheat-sheet

- `office_recommended_goals` : id, office_id, recommended_goal_cents, rationale, source_url, created_at, updated_at
- `goal_increase_requests` : id, wouldbe_id, requested_by_user_id, current_goal_cents, requested_goal_cents, reason, status, review_method, reviewed_by_user_id, reviewed_at, review_notes, created_at

10. Pledges

Route file: `server/API/pledgesRoutes.js` · **DB file:** `server/DB/pledges.js` (new) **Tables in play:** `pledges` (+ `wouldbe` , `plan_timeline_components` , `user_attestations` , `wouldbe_cap_resets`)

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/wouldbes/:id/pledges</code>	<code>isPledgeEligible</code> (<i>gate</i>) → <code>createPledge</code> (<i>stamps plan_timeline_component_id = open stage; rejects over-cap via getRemainingPledgeCap</i>)	<code>pledges</code> W · <code>user_attestations</code> R · <code>wouldbe</code> R · <code>wouldbe_cap_resets</code> R	A, ATT(<code>us_citizen_or_lpr</code>)	☐
GET <code>/pledges/me</code>	<code>getUserPledges</code>	<code>pledges</code> R	A	☐
POST <code>/pledges/:id/withdraw</code>	<code>withdrawPledge</code>	<code>pledges</code> W	A	☐
GET <code>/wouldbes/:id/pledge-stats</code>	<code>getWouldbePledgeStats</code>	<code>pledges</code> R	—	☐
(<i>lifecycle</i>)	<code>markPledgeConverted</code> · <code>markPledgesGoalReached</code> (→ <i>emit pledge_goal_reached</i> ; <i>pairs with [DB] recordGoalReached</i>) · <code>resetPledgeCapForPrimaryAdvance</code>	<code>pledges</code> W · <code>wouldbe_cap_resets</code> W	internal	☐

Note: pledges are **promises** — they never transact on-platform. The contribution itself happens at the external processor (the `[DB]` ActBlue/WinRed link). Platform revenue is `tips` (§13).

Column cheat-sheet

- `pledges` : id, pledger_user_id, wouldbe_id, amount_cents, status, converted_at, cap_window, plan_timeline_component_id, created_at
- `wouldbe_cap_resets` : id, wouldbe_id, reset_number, reason, additional_cap_cents, confirmed_by, proof_reference, confirmed_at, created_at

11. Follows

Route file: `server/API/followsRoutes.js` · **DB file:** `server/DB/follows.js` (new) **Tables in play:** `follows` (+ `users` , `wouldbe` for the feed)

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/follows</code>	<code>createFollow</code>	<code>follows</code> W	A	☐
DELETE <code>/follows/:id</code>	<code>deleteFollow</code>	<code>follows</code> W	A	☐
GET <code>/users/:id/followers</code>	<code>getFollowers</code>	<code>follows</code> R · <code>users</code> R	—	☐
GET <code>/users/:id/following</code>	<code>getFollowing</code>	<code>follows</code> R · <code>users</code> R	—	☐
GET <code>/feed</code>	<code>getHomeFeed</code>	<code>follows</code> R · <code>wouldbe</code> R · <code>posts</code> R	A	☐

Column cheat-sheet

- `follows` : `id`, `follower_id`, `followed_id`, `follow_type`, `created_at`

§6 — Sponsors, Debates, Rules, Criteria, Judges — *none built*

Plus the Debate Update's **livestream subsystem** and **sponsor-fee economics**.

12. Sponsors

Route file: `server/API/sponsorsRoutes.js` · **DB file:** `server/DB/sponsors.js` (new) **Tables in play:** `sponsors` (+ `users` , `debates`)

Method / Path	Function	Tables (R/W)	MW	Status
POST <code>/sponsors</code>	<code>createSponsor</code>	<code>sponsors</code> W	A	☐
GET <code>/sponsors/:id</code>	<code>getSponsorById</code>	<code>sponsors</code> R	—	☐
PATCH <code>/sponsors/:id</code>	<code>updateSponsor</code> · [DB] <code>setSponsorMarketingConsent</code> (<code>marketing_consent_at</code>)	<code>sponsors</code> W	A	☐
POST <code>/sponsors/:id/verify</code>	<code>verifySponsor</code>	<code>sponsors</code> W	AD	☐
GET <code>/sponsors/:id/debates</code>	<code>getSponsorDebates</code>	<code>debates</code> R	—	☐

Column cheat-sheet

- `sponsors` : `id`, `user_id`, `type`, `display_name`, `logo_url`, `verified_at`, [DB] `marketing_consent_at`, `created_at`, `updated_at`

13. Debates (incl. [DB] *sponsor-fee economics*)

Route file: `server/API/debatesRoutes.js` · **DB file:** `server/DB/debates.js` (new) **Tables in play:** `debates` (+ `sponsors` , `contestants` , `debate_votes` ; [DB] `debate_payments` for the flat fee/entry split)

Method / Path	Function	Tables (R/W)	MW	Status			
POST <code>/debates</code>	<code>createDebate</code> · [DB] <code>set sponsor_flat_fee_cents</code> (post fee) + <code>sponsor-set sponsor_entry_fee_cents</code> → <code>entry_price_cents</code> (fee + processing markup); <code>concluding_stream_at</code>	<code>debates</code> W	A	☐			
GET <code>/debates</code>	<code>listDebates</code>	<code>debates</code> R	—	☐			
GET <code>/debates/:id</code>	<code>getDebateById</code>	<code>debates</code> R	—	☐			
PATCH <code>/debates/:id</code>	<code>updateDebate</code> · [DB] <code>setDebateMarketingConsent</code>	<code>debates</code> W	A	☐			
POST <code>/debates/:id/publish</code>	<code>start</code>	<code>close</code>	<code>cancel</code>	<code>publishDebate</code> · <code>startDebate</code> · <code>closeDebate</code> · <code>cancelDebate</code>	<code>debates</code> W	A	☐
GET <code>/debates/:id/leaderboard</code>	<code>getDebateLeaderboard</code>	<code>debate_votes</code> R · <code>contestants</code> R	—	☐			
[DB] POST <code>/debates/:id/sponsor-fee</code>	<code>recordSponsorFlatFeePayment</code> (<code>payment_type='debate_sponsor_flat_fee'</code>)	<code>debate_payments</code> W	A	☐			
(internal)	<code>incrementDebateContributions</code>	<code>debates</code> W	internal	☐			

Column cheat-sheet

- `debates` : id, sponsor_id, title, description, win_type, hybrid_crowd_weight_pct, contribution_type, participation_type, sponsor_contribution_cents, platform_top_up_cents, user_contributions_cents, prize_pool_cents, prize_distribution_rules, scoring_methodology, status, start_date, end_date, results_announce_at, min_age_required, excluded_states, free_entry_method, retired, [DB] sponsor_flat_fee_cents, sponsor_entry_fee_cents, entry_price_cents, concluding_stream_at, marketing_consent_at, created_at, updated_at
- [DB] `debate_payments` (§13): payment_id, user_id, debate_id, payment_type (+ `debate_sponsor_flat_fee`), amount_cents, currency, fee_amount_cents, net_amount_cents, [DB] sponsor_amount_cents, platform_amount_cents, stripe_*, status, charged_at, refunded_at, created_at

14. Debate rules / criteria / judges

Route files: `server/API/debateRulesRoutes.js` · `debateCriteriaRoutes.js` · `debateJudgesRoutes.js` · DB files: `debateRules.js` · `debateCriteria.js` · `debateJudges.js` (new) Tables in play: `debate_official_rules` , `debate_judging_criteria` , `debate_judges` , `debate_judge_scores` (+ `contestants`)

Method / Path	Function	Tables (R/W)	MW	Status
POST /admin/debates/:id/rules	<code>publishDebateRules</code> (versioned; closes prior)	<code>debate_official_rules</code> W	AD	☐
GET /debates/:id/rules	<code>getCurrentDebateRules</code> · /rules/history <code>getDebateRulesHistory</code> · /rules/:version <code>getRulesByVersion</code>	<code>debate_official_rules</code> R	—	☐
POST /debates/:id/criteria	<code>addCriterion</code> then <code>validateCriteriaWeightsSum</code>	<code>debate_judging_criteria</code> W	AD	☐
GET /debates/:id/criteria	<code>getDebateCriteria</code> · PATCH <code>updateCriterion</code> · DELETE <code>deleteCriterion</code>	<code>debate_judging_criteria</code> R/W	AD	☐
POST /debates/:id/judges	<code>addJudge</code> · GET <code>getDebateJudges</code> · POST /judges/:id/recuse <code>recuseJudge</code>	<code>debate_judges</code> W	AD	☐
POST /judges/:id/scores	<code>submitJudgeScores</code> then <code>lockJudgeScores</code> · GET /debates/:id/judge-scores <code>getDebateJudgeScores</code>	<code>debate_judge_scores</code> W · <code>contestants</code> R	A	☐

Column cheat-sheet

- `debate_official_rules` : rules_id, debate_id, version, rules_text, rules_url, body_hash, effective_at, superseded_at, age_eligibility_min, age_eligibility_max, minor_entry_allowed, minor_entry_methods, created_at
- `debate_judging_criteria` : criterion_id, debate_id, criterion_key, display_name, description, weight, display_order, created_at
- `debate_judges` : judge_id, debate_id, user_id, external_name, external_bio, role, disclosed_at, recused_at, recusal_reason, created_at
- `debate_judge_scores` : score_id, debate_id, judge_id, contestant_id, criterion_id, score, notes, locked_at, created_at, updated_at

15. [DB] Twitch livestream subsystem

Route files: `server/API/twitchRoutes.js` · `debateStreamsRoutes.js` · DB files: `twitch.js` · `debateStreams.js` (new) Tables in play: `twitch_connections` , `debate_streams` , `twitch_eventsub_subscriptions` , `twitch_eventsub_events` , `stream_recordings` , `stream_participant_consent`s

Method / Path	Function	Tables (R/W)	MW	Status
GET /twitch/oauth/start · /callback	upsertTwitchConnection (capture channel + user id, store token, record consent)	twitch_connections W	A	☐
(onboarding)	verifyVodStorage (Get Videos after first stream → vod_storage_verified_at)	twitch_connections W	A	☐
POST /admin/twitch/eventsub	createEventSubSubscription · deleteEventSubSubscription	twitch_eventsub_subscriptions W	AD	☐
POST /internal/twitch/eventsub (webhook)	handleEventSubWebhook (verify HMAC; dedupe message_id) → markStreamLive / markStreamOffline	twitch_eventsub_events W · debate_streams W	internal	☐
POST /debates/:id/stream	scheduleDebateStream *(method=embed\	hybrid_record; embed_parent_domains)*	debate_streams W	A/AD ☐
GET /debates/:id/stream	getDebateStream	debate_streams R	—	☐
(Method 2, internal)	ingestStreamRecording / attachStreamerVod · setRecordingModerationStatus · publishRecording · purgeExpiredRecordings (auto-delete window)	stream_recordings W	internal	☐
POST /debate-streams/:id/consent	recordStreamParticipantConsent (hosting/replay, marketing, group-stream, minor)	stream_participant_consents W	A, CTX	☐

Column cheat-sheet

- twitch_connections** : id, user_id, twitch_user_id, twitch_login, twitch_display_name, access_token, refresh_token, scopes, token_expires_at, developer_tos_accepted_at, vod_storage_verified_at, connected_at, disconnected_at, updated_at (encrypt tokens at rest)
- debate_streams** : id, debate_id, twitch_connection_id, host_user_id, method, twitch_channel, twitch_broadcaster_user_id, embed_parent_domains, scheduled_at, status, started_at, ended_at, vod_video_id, vod_url, created_at, updated_at
- twitch_eventsub_subscriptions** : id, twitch_connection_id, subscription_id, subscription_type, broadcaster_user_id, status, created_at, updated_at
- twitch_eventsub_events** : id, message_id, subscription_type, broadcaster_user_id, debate_stream_id, signature_verified, event_payload, received_at
- stream_recordings** : id, debate_stream_id, debate_id, source, r2_bucket, r2_object_key, playback_url, duration_seconds, moderation_status, published_at, auto_delete_at, deleted_at, created_at, updated_at
- stream_participant_consents** : id, debate_stream_id, user_id, role, hosting_replay_license_at, marketing_release_at, group_stream_consent_at, is_minor, guardian_consent_at, consent_document_version, ip_address, user_agent, created_at

16. [DB] Sponsor payouts from entries

Route file: `server/API/debatePayoutsRoutes.js` · **DB file:** `server/DB/debatePayouts.js` (new) **Tables in play:** `debate_sponsor_payouts` (+ `debate_payments` to sum entries; winner payout = `prize_distributions`, §9)

Method / Path	Function	Tables (R/W)	MW	Status
(on conclusion)	computeSponsorEntryPayout (sum debate_payments.sponsor_amount_cents) → disburseSponsorPayout (Stripe transfer)	debate_sponsor_payouts W · debate_payments R	AD, ADA	☐
GET /debates/:id/sponsor-payouts	getSponsorPayouts	debate_sponsor_payouts R	AD	☐

Column cheat-sheet

- debate_sponsor_payouts** : id, debate_id, sponsor_id, recipient_user_id, gross_entries_cents, platform_fee_cents, amount_cents, currency, stripe_transfer_id, disbursement_method, status, disbursed_at, created_at, updated_at

Quick mental model

- \$4 gates \$5.** A WouldBe can't go `active` until `hasActiveVerifiedCommittee` (committee) passes; `compliance_checks` / `fundraising_eligibility_checks` read the versioned `jurisdiction_rules_versions` to decide. Money never moves here.
- \$5 producer chain.** `computeDeadlinesForOffice` → `election_deadlines` → `buildStagedTimeline` splits `wouldbe.goal_cents` into per-deadline **micro-goals** (`plan_timeline_components.sub_goal_cents`). Pledges attribute to the open stage; when a goal/micro-goal is met, [DB] `recordGoalReached` fans the **ActBlue/WinRed** link to pledgers. Pledges are promises — the contribution happens off-platform; platform revenue is `tips`.

- **\$6 money.** Sponsor pays a **flat fee to post** (`sponsor_flat_fee_cents` , platform revenue) + the cash prize. Entries are `sponsor_entry_fee_cents` + processing markup = `entry_price_cents` ; entry fees pay out to the **sponsor** (`debate_sponsor_payouts`), the prize to the **winner** (`prize_distributions`), both once the contest concludes.
- **\$6 livestream.** Twitch OAuth per streamer (`twitch_connections`) → EventSub `stream.online/offline` (`twitch_eventsub_*`) flips `debate_streams.status` → Method `stream.online/offline` embeds; Method `twitch_eventsub_*` also keeps an R2 copy (`stream_recordings`) it moderates then auto-deletes. Every participant signs `stream_participant_consent`s .